

AMRITA VISHWA VIDYAPEETHAM CHENNAI
CAMPUS

Amrita School of Computing

Coding Practice

Activity Sheets

(For the Batch 2023-2027)

Name: NAGARAMPALLI SARVAN KUMAR

Roll No.: CH.SC.U4CSE24130

Department: CSE - CT

Instructions

The **Mock Test syllabus** for CSE CT | CYS | AIE | RAI program is enclosed in the attached document for your reference.

Materials for the Mock Test [Must-DO Coding Questions] is provided in a link.

It is mandatory to upload the **completed Activity sheets** before the deadline in the MS-Form link shared by your class Advisor.

Activity Sheet: 01 - Coding Questions from Arrays – Part 1 [Easy + Medium Questions]

Each Activity Sheets contains total of **20 Questions**.

Platform: Any Online Compiler of your choice – **GDB Compiler**

Language: C | C++ | JAVA | PYTHON

The Activity Sheet will have the coding Questions and the Test Case. Once executed in Online Compiler, take **screen shot of the code and the output** and paste it in the space provided in the activity sheet.

The Final Activity Sheet should be in **PDF Format** and the file name should be your **roll number**.

Course Instructors: CARE TEAM

Dr.R.Annamalai	Assistant Professor (Sl.Grade) CSE-AIE
Dr.J.Umamageswaran	Assistant Professor (Sr.Grade) CSE-CT
Ms.D. Sasikala	Assistant Professor CSE-AIE
Mr.NirmalRaj	Assistant Professor CSE-CYS

1. Given an array of integers `arr[]` representing non-negative integers, arrange them so that after concatenating all of them in order, it results in the largest possible number. Since the result may be very large, return it as a string.

Examples:

Input: `arr[] = [3, 30, 34, 5, 9]`

Output: 9534330

Explanation: Given numbers are [3, 30, 34, 5, 9], the arrangement [9, 5, 34, 3, 30] gives the largest value.

Input: `arr[] = [54, 546, 548, 60]`

Output: 6054854654

Explanation: Given numbers are [54, 546, 548, 60], the arrangement [60, 548, 546, 54] gives the largest value.

Input: `arr[] = [3, 4, 6, 5, 9]`

Output: 96543

Explanation: Given numbers are [3, 4, 6, 5, 9], the arrangement [9, 6, 5, 4, 3] gives the largest value.

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$0 \leq \text{arr}[i] \leq 10^5$

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

bool cmp(string a, string b){
    return a + b > b + a;
}

int main() {
    int n;
    cin >> n;
    vector<string> arr(n);

    for(int i = 0; i < n; i++){
        int x;
        cin >> x;
        arr[i] = to_string(x);
    }

    sort(arr.begin(), arr.end(), cmp);

    if(arr[0] == "0"){
        cout << "0";
        return 0;
    }

    for(string s : arr) cout << s;
    return 0;
}
```

Output:

```
● PS S:\care\arrays_sheet_1> g++ Q1_LargestNumber.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
5
3 30 34 5 9
9534330
❖ PS S:\care\arrays_sheet_1> █
```

2. A celebrity is a person who is known to all but does not know anyone at a party. A party is being organized by some people. A square matrix `mat[][]` of size $n \times n$ is used to represent people at the party such that if an element of row i and column j is set to 1 it means i th person knows j th person. You need to return the index of the celebrity in the party, if the celebrity does not exist, return -1.

Note: Follow 0-based indexing.

Examples:

Input: `mat[][] = [[1, 1, 0],`
 `[0, 1, 0],`
 `[0, 1, 1]]`

Output: 1

Explanation: 0th and 2nd person both know 1st person and 1st person does not know anyone. Therefore, 1 is the celebrity person.

Input: `mat[][] = [[1, 1],`
 `[1, 1]]`

Output: -1

Explanation: Since both the people at the party know each other. Hence none of them is a celebrity person.

Input: `mat[][] = [[1]]`

Output: 0

Constraints:

$1 \leq \text{mat.size()} \leq 1000$

$0 \leq \text{mat}[i][j] \leq 1$

$\text{mat}[i][i] = 1$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<vector<int>> mat(n, vector<int>(n));

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            cin >> mat[i][j];

    int celeb = 0;

    for(int i = 1; i < n; i++){
        if(mat[celeb][i] == 1)
            celeb = i;
    }

    for(int i = 0; i < n; i++){
        if(i != celeb && (mat[celeb][i] == 1 || mat[i][celeb] == 0)){
            cout << -1;
            return 0;
        }
    }

    cout << celeb;
    return 0;
}
```

Output

```
● PS S:\care\arrays_sheet_1> g++ Q2_CelebrityProblem.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
2
1 1
1 1
-1
❏ PS S:\care\arrays_sheet_1> █
```


3. Given an unsorted array `arr[]`, find the first element such that every element to its left is less than or equal to it, and every element to its right is greater than or equal to it.

Note: If no such element exists, return -1.

Examples :

Input: `arr = [4, 2, 5, 7]`

Output: 5

Explanation: All elements to the left of 5 are less than or equal to 5, and all elements to the right are greater than or equal to 5.

Input: `arr = [11, 9, 12]`

Output: -1

Explanation: No element in the array satisfies the required condition.

Constraints:

$3 \leq \text{arr.size()} \leq 10^6$

$1 \leq \text{arr}[i] \leq 10^6$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n), leftMax(n), rightMin(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    leftMax[0] = arr[0];
    for(int i = 1; i < n; i++)
        leftMax[i] = max(leftMax[i-1], arr[i]);

    rightMin[n-1] = arr[n-1];
    for(int i = n-2; i >= 0; i--)
        rightMin[i] = min(rightMin[i+1], arr[i]);

    for(int i = 0; i < n; i++){
        if(arr[i] >= leftMax[i] && arr[i] <= rightMin[i]){
            cout << arr[i];
            return 0;
        }
    }

    cout << -1;
    return 0;
}
```

Output

```
● PS S:\care\arrays_sheet_1> g++ Q3_PivotElement.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
4
4 2 5 7
5
❖ PS S:\care\arrays_sheet_1> █
```

4. Given an array of positive integers. Your task is to rearrange the array elements alternatively i.e. first element should be the max value, the second should be the min value, the third should be the second max, the fourth should be the second min, and so on.

Note: Modify the original array itself. Do it without using any extra space. You do not have to return anything.

Examples:

Input: arr[] = [1, 2, 3, 4, 5, 6]

Output: [6, 1, 5, 2, 4, 3]

Explanation: Max element = 6, min = 1, second max = 5, second min = 2, and so on... The modified array is: [6, 1, 5, 2, 4, 3]

Input: arr[] = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110]

Output: [110, 10, 100, 20, 90, 30, 80, 40, 70, 50, 60]

Explanation: Max element = 110, min = 10, second max = 100, second min = 20, and so on... Modified array is : [110, 10, 100, 20, 90, 30, 80, 40, 70, 50, 60]

Input: arr[] = [1]

Output: [1]

Constraints:

$1 \leq \text{arr.size} \leq 10^6$

$1 \leq \text{arr}[i] \leq 10^6$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    vector<int> ans;
    int l = 0, r = n - 1;

    while(l <= r){
        if(l != r){
            ans.push_back(arr[r--]);
            ans.push_back(arr[l++]);
        } else {
            ans.push_back(arr[l++]);
        }
    }

    for(int x : ans) cout << x << " ";
    return 0;
}
```

Output

```
● PS S:\care\arrays_sheet_1> g++ Q4_RearrangeMaxMin.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
6
1 2 3 4 5 6
6 1 5 2 4 3
❖ PS S:\care\arrays_sheet_1> |
```

5. You are given a sorted array `arr[]` containing positive integers. Your task is to remove all duplicate elements from this array such that each element appears only once. Return an array containing these distinct elements in the same order as they appeared.

Examples :

Input: `arr[] = [2, 2, 2, 2, 2]`

Output: `[2]`

Explanation: After removing all the duplicates only one instance of 2 will remain i.e. `[2]` so modified array will contain 2 at first position and you should return array containing `[2]` after modifying the array.

Input: `arr[] = [1, 2, 4]`

Output: `[1, 2, 4]`

Explanation: As the array does not contain any duplicates so you should return `[1, 2, 4]`.

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^6$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    cout << arr[0] << " ";

    for(int i = 1; i < n; i++){
        if(arr[i] != arr[i-1])
            cout << arr[i] << " ";
    }

    return 0;
}
```


Output

```
• PS S:\care\arrays_sheet_1> g++ Q5_RemoveDuplicatesSorted.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
5
2 2 2 2 2
2
PS S:\care\arrays_sheet_1> █
```

6. Given two integer arrays a and b, the task is to find the numbers which are present in the first array a, but not present in the second array b.

Note: Numbers to be returned should in order as they appear in array a.

Examples :

Input: a[] = [1, 2, 3, 4, 5, 10], b[] = [2, 3, 1, 0, 5]

Output: [4, 10]

Explanation: 4 and 10 are present in first array, but not in second array.

Input: a[] = [4, 3, 5, 9, 11], b[] = [4, 9, 3, 11, 10]

Output: [5]

Explanation: Second array does not contain element 5.

Input: a[] = [9], b[] = [7, 9, 4, 9, 9, 9]

Output: []

Constraints:

$1 \leq a.size(), b.size() \leq 10^5$

$1 \leq a[i] \leq 10^5$

$1 \leq b[i] \leq 10^5$

Code:

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

int main() {
    int n, m;
    cin >> n;
    vector<int> a(n);
    for(int i = 0; i < n; i++) cin >> a[i];

    cin >> m;
    vector<int> b(m);
    unordered_set<int> s;

    for(int i = 0; i < m; i++){
        cin >> b[i];
        s.insert(b[i]);
    }

    for(int x : a){
        if(!s.count(x))
            cout << x << " ";
    }

    return 0;
}
```

Output

```
● PS S:\care\arrays_sheet_1> g++ Q6_ArrayDifference.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
6
1 2 3 4 5 10
5
2 3 1 0 5
4 10
❖ PS S:\care\arrays_sheet_1> 
```

7. Given an array `arr[]` consisting of positive integers, return the array by removing all duplicate numbers.

Example:

Input: `arr[] = [2, 2, 3, 3, 7, 5]`

Output: `[2, 3, 7, 5]`

Explanation: After removing the duplicates 2 and 3 we get 2 3 7 5.

Input: `arr[] = [1, 2, 3, 4, 5]`

Output: `[1, 2, 3, 4, 5]`

Explanation: There doesn't exist any duplicate element.

Constraints:

$2 \leq \text{arr.size()} \leq 10^6$

$1 \leq \text{arr}[i] \leq 10^9$

Code:

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n);
    unordered_set<int> seen;

    for(int i = 0; i < n; i++) cin >> arr[i];

    for(int x : arr){
        if(!seen.count(x)){
            cout << x << " ";
            seen.insert(x);
        }
    }

    return 0;
}
```

Output

```
● PS S:\care\arrays_sheet_1> g++ Q7_RemoveDuplicatesUnsorted.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
6
2 2 3 3 7 5
2 3 7 5
❖ PS S:\care\arrays_sheet_1> |
```

8. You have given two sorted arrays `a[]` & `b[]` of distinct elements. The first array has one element extra added in between. Return the index of the extra element.

Note: 0-based indexing is followed.

Examples

Input: `a[] = [2,4,6,8,9,10,12]`, `b[] = [2,4,6,8,10,12]`

Output: 4

Explanation: In the first array, 9 is extra added and it's index is 4.

Input: `a[] = [3,5,7,8,11,13]`, `b[] = [3,5,7,11,13]`

Output: 3

Explanation: In the first array, 8 is extra and it's index is 3.

Input: `a[] = [3,5]`, `b[] = [3]`

Output: 1

Explanation: In the first array, 5 is extra and it's index is 1.

Constraints:

$2 \leq \text{arr1.size()} \leq 10^5$

$1 \leq \text{arr1}[i], \text{arr2}[i] \leq 10^6$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> a(n), b(n-1);

    for(int i = 0; i < n; i++) cin >> a[i];
    for(int i = 0; i < n-1; i++) cin >> b[i];

    int low = 0, high = n - 2, ans = n - 1;

    while(low <= high){
        int mid = (low + high) / 2;

        if(a[mid] == b[mid])
            low = mid + 1;
        else{
            ans = mid;
            high = mid - 1;
        }
    }

    cout << ans;
    return 0;
}
```

Output

```
• PS S:\care\arrays_sheet_1> g++ Q8_ExtraElementIndex.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
7
2 4 6 8 9 10 12
2 4 6 8 10 12
4
PS S:\care\arrays_sheet_1> █
```

9. Given an unsorted array `arr[]`, find the first element such that every element to its left is less than or equal to it, and every element to its right is greater than or equal to it.

Note: If no such element exists, return -1.

Examples :

Input: `arr = [4, 2, 5, 7]`

Output: 5

Explanation: All elements to the left of 5 are less than or equal to 5, and all elements to the right are greater than or equal to 5.

Input: `arr = [11, 9, 12]`

Output: -1

Explanation: No element in the array satisfies the required condition.

Constraints:

$3 \leq \text{arr.size()} \leq 10^6$

$1 \leq \text{arr}[i] \leq 10^6$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> arr(n), left(n), right(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    left[0] = arr[0];
    for(int i = 1; i < n; i++)
        left[i] = max(left[i-1], arr[i]);

    right[n-1] = arr[n-1];
    for(int i = n-2; i >= 0; i--)
        right[i] = min(right[i+1], arr[i]);

    for(int i = 0; i < n; i++){
        if(arr[i] >= left[i] && arr[i] <= right[i]){
            cout << arr[i];
            return 0;
        }
    }

    cout << -1;
    return 0;
}
```

Output

```
• PS S:\care\arrays_sheet_1> g++ Q9_PivotElementAgain.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
4
4 2 5 7
5
❏ PS S:\care\arrays_sheet_1> █
```

10. Given an array `arr[]` of integers and another integer `target`. Determine if there exist two distinct indices such that the sum of their elements is equal to the target.

Examples:

Input: `arr[] = [0, -1, 2, -3, 1]`, `target = -2`

Output: true

Explanation: `arr[3] + arr[4] = -3 + 1 = -2`

Input: `arr[] = [1, -2, 1, 0, 5]`, `target = 0`

Output: false

Explanation: None of the pair makes a sum of 0

Input: `arr[] = [11]`, `target = 11`

Output: false

Explanation: No pair is possible as only one element is present in `arr[]`

Constraints:

$1 \leq \text{arr.size} \leq 10^5$

$-10^5 \leq \text{arr}[i] \leq 10^5$

$-2*10^5 \leq \text{target} \leq 2*10^5$

Code:

```
#include <iostream>
#include <vector>
#include <unordered_set>
using namespace std;

int main() {
    int n, target;
    cin >> n;

    vector<int> arr(n);
    for(int i = 0; i < n; i++) cin >> arr[i];

    cin >> target;

    unordered_set<int> s;

    for(int x : arr){
        if(s.count(target - x)){
            cout << "true";
            return 0;
        }
        s.insert(x);
    }

    cout << "false";
    return 0;
}
```

Output

```
• PS S:\care\arrays_sheet_1> g++ Q10_PairSum.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
5
0 -1 2 -3 1
-2
true
❖ PS S:\care\arrays_sheet_1> |
```


11. Given an array arr the task is to sort all even-placed numbers in increasing and odd-place numbers in decreasing order. The modified array should contain all sorted even-placed numbers followed by reverse sorted odd-placed numbers.

Note: that the first element is considered as even because of its index 0.

Examples:

Input: arr[] = [0, 1, 2, 3, 4, 5, 6, 7]

Output: [0, 2, 4, 6, 7, 5, 3, 1]

Explanation: Even-place elements : 0, 2, 4, 6 Odd-place elements : 1, 3, 5, 7 Even-place elements in increasing order : 0, 2, 4, 6 Odd-Place elements in decreasing order : 7, 5, 3, 1

Input: arr[] = [3, 1, 2, 4, 5, 9, 13, 14, 12]

Output: [2, 3, 5, 12, 13, 14, 9, 4, 1]

Explanation: Even-place elements : 3, 2, 5, 13, 12 Odd-place elements : 1, 4, 9, 14 Even-place elements in increasing order : 2, 3, 5, 12, 13 Odd-Place elements in decreasing order : 14, 9, 4, 1

Expected Auxiliary Space: $O(n \log n)$

Expected Auxiliary Space: $O(n)$

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^5$

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> arr(n), even, odd;

    for(int i = 0; i < n; i++){
        cin >> arr[i];

        if(i % 2 == 0) even.push_back(arr[i]);
        else odd.push_back(arr[i]);
    }

    sort(even.begin(), even.end());
    sort(odd.rbegin(), odd.rend());

    for(int x : even) cout << x << " ";
    for(int x : odd) cout << x << " ";

    return 0;
}
```

Output

```
PS S:\care\arrays_sheet_1> g++ Q11_EvenOddPlaceSort.cpp
PS S:\care\arrays_sheet_1> ./a.exe
8
0 1 2 3 4 5 6 7
0 2 4 6 7 5 3 1
PS S:\care\arrays_sheet_1> 
```

12. Given an array `arr`, return the modified array in such a way that if the current and next numbers are not zero and are equal then double the current and replace the next number with 0. After the modification, rearrange the array such that all 0's are shifted to the end.

Note: The sequence of the non-zeros should in the same order in output.

Examples:

Input: `arr[] = [2, 2, 0, 4, 0, 8]`

Output: `[4, 4, 8, 0, 0, 0]`

Explanation: At index 0 and 1 both the elements are the same. So, we change the element at index 0 to 4 and the element at index 1 is 0 then shift all the zeros to the end of the array. So, the array become `[4, 4, 8, 0, 0, 0]`. After this there is no further change as the remaining array elements are either 0 or not same.

Input: `arr[] = [0, 2, 2, 2, 0, 6, 6, 0, 0, 8]`

Output: `[4, 2, 12, 8, 0, 0, 0, 0, 0, 0]`

Explanation: Since the elements at indexes 1 to 2 are same, we change array to `[4, 2, 6, 6, 8, 0, 0, 0, 0, 0]`. Now we have two 6s same, so array becomes `[4, 2, 12, 8, 0, 0, 0, 0, 0, 0]`

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^6$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> arr(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    for(int i = 0; i < n-1; i++){
        if(arr[i] != 0 && arr[i] == arr[i+1]){
            arr[i] *= 2;
            arr[i+1] = 0;
        }
    }

    vector<int> ans;

    for(int x : arr)
        if(x != 0) ans.push_back(x);

    while(ans.size() < n)
        ans.push_back(0);

    for(int x : ans) cout << x << " ";

    return 0;
}
```

Output

```
• PS S:\care\arrays_sheet_1> g++ Q12_DoubleAndShiftZeros.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
6
2 2 0 4 0 8
4 4 8 0 0 0
❖ PS S:\care\arrays_sheet_1> |
```

13. You are given an array `arr[]` containing positive integers. The elements in the array `arr[]` range from 1 to `n` (where `n` is the size of the array), and some numbers may be repeated or absent. You have to count the frequency of all numbers in the range 1 to `n` and return an array of size `n` such that `result[i]` represents the frequency of the number `i` (1-based indexing).

Examples:

Input: `arr[] = [2, 3, 2, 3, 5]`

Output: `[0, 2, 2, 0, 1]`

Explanation: We have: 1 occurring 0 times, 2 occurring 2 times, 3 occurring 2 times, 4 occurring 0 times, and 5 occurring 1 time.

Input: `arr[] = [3, 3, 3, 3]`

Output: `[0, 0, 4, 0]`

Explanation: We have: 1 occurring 0 times, 2 occurring 0 times, 3 occurring 4 times, and 4 occurring 0 times.

Input: `arr[] = [1]`

Output: `[1]`

Explanation: We have: 1 occurring 1 time, and there are no other numbers between 1 and the size of the array.

Constraints:

$1 \leq \text{arr.size()} \leq 10^6$

$1 \leq \text{arr}[i] \leq \text{arr.size()}$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> arr(n), freq(n + 1, 0);

    for(int i = 0; i < n; i++){
        cin >> arr[i];
        freq[arr[i]]++;
    }

    for(int i = 1; i <= n; i++)
        cout << freq[i] << " ";

    return 0;
}
```


Output

```
● PS S:\care\arrays_sheet_1> g++ Q13_FrequencyCount.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
5
2 3 2 3 5
0 2 2 0 1
❖ PS S:\care\arrays_sheet_1> █
```

14. Given a data stream `arr[]` where integers are read sequentially, the task is to determine the median of the elements encountered so far after each new integer is read.

The median is defined as follows:

1. Odd number of elements: The median is the middle element when the current set of numbers is sorted.
2. Even number of elements: The median is the arithmetic mean (average) of the two middle elements when the current set of numbers is sorted.

Examples:

Input: `arr[] = [5, 15, 1, 3, 2, 8]`

Output: `[5.0, 10.0, 5.0, 4.0, 3.0, 4.0]`

Explanation:

After reading 1st element of stream – 5 -> median = 5.0

After reading 2nd element of stream – 5, 15 -> median = $(5+15)/2 = 10.0$

After reading 3rd element of stream – 5, 15, 1 -> median = 5.0

After reading 4th element of stream – 5, 15, 1, 3 -> median = $(3+5)/2 = 4.0$

After reading 5th element of stream – 5, 15, 1, 3, 2 -> median = 3.0

After reading 6th element of stream – 5, 15, 1, 3, 2, 8 -> median = $(3+5)/2 = 4.0$

Input: `arr[] = [2, 2, 2, 2]`

Output: `[2.0, 2.0, 2.0, 2.0]`

Explanation:

After reading 1st element of stream – 2 -> median = 2.0

After reading 2nd element of stream – 2, 2 -> median = $(2+2)/2 = 2.0$

After reading 3rd element of stream – 2, 2, 2 -> median = 2.0

After reading 4th element of stream – 2, 2, 2, 2 -> median = $(2+2)/2 = 2.0$

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^6$

Code:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    priority_queue<int> left;
    priority_queue<int, vector<int>, greater<int>> right;
    for(int i = 0; i < n; i++){
        int x;
        cin >> x;
        if(left.empty() || x <= left.top())
            left.push(x);
        else
            right.push(x);

        if(left.size() > right.size() + 1){
            right.push(left.top());
            left.pop();
        }

        if(right.size() > left.size()){
            left.push(right.top());
            right.pop();
        }
        double median;
        if(left.size() == right.size())
            median = (left.top() + right.top()) / 2.0;
        else
            median = left.top();
        cout << fixed << setprecision(1) << median << " ";
    }
    return 0;
}
```

Output:

```
● PS S:\care\arrays_sheet_1> g++ Q14_RunningMedian.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
6
5 15 1 3 2 8
5.0 10.0 5.0 4.0 3.0 4.0
❖ PS S:\care\arrays_sheet_1> █
```

15. Given an array `arr` of 0s and 1s. Find and return the length of the longest subarray with equal number of 0s and 1s.

Examples:

Input: `arr[] = [1, 0, 1, 1, 1, 0, 0]`

Output: 6

Explanation: `arr[1...6]` is the longest subarray with three 0s and three 1s.

Input: `arr[] = [0, 0, 1, 1, 0]`

Output: 4

Explanation: `arr[0...3]` or `arr[1...4]` is the longest subarray with two 0s and two 1s.

Input: `arr[] = [0]`

Output: 0

Explanation: There is no subarray with an equal number of 0s and 1s.

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$0 \leq \text{arr}[i] \leq 1$

Code:

```
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> arr(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    unordered_map<int,int> mp;
    mp[0] = -1;

    int sum = 0, ans = 0;

    for(int i = 0; i < n; i++){
        sum += (arr[i] == 0 ? -1 : 1);

        if(mp.count(sum))
            ans = max(ans, i - mp[sum]);
        else
            mp[sum] = i;
    }

    cout << ans;
    return 0;
}
```

Output

```
• PS S:\care\arrays_sheet_1> g++ Q15_LongestEqual01.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
7
1 0 1 1 1 0 0
6
❖ PS S:\care\arrays_sheet_1> 
```

16. Given an integer array `coins[ ]` representing different denominations of currency and an integer `sum`, find the number of ways you can make `sum` by using different combinations from `coins[ ]`.

Note: Assume that you have an infinite supply of each type of coin. Therefore, you can use any coin as many times as you want.

Answers are guaranteed to fit into a 32-bit integer.

Examples:

Input: `coins[] = [1, 2, 3]`, `sum = 4`

Output: 4

Explanation: Four Possible ways are: [1, 1, 1, 1], [1, 1, 2], [2, 2], [1, 3].

Input: `coins[] = [2, 5, 3, 6]`, `sum = 10`

Output: 5

Explanation: Five Possible ways are: [2, 2, 2, 2, 2], [2, 2, 3, 3], [2, 2, 6], [2, 3, 5] and [5, 5].

Input: `coins[] = [5, 10]`, `sum = 3`

Output: 0

Explanation: Since all coin denominations are greater than `sum`, no combination can make the target `sum`.

Constraints:

$1 \leq \text{sum} \leq 10^3$

$1 \leq \text{coins}[i] \leq 10^4$

$1 \leq \text{coins.size()} \leq 10^3$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, sum;
    cin >> n;

    vector<int> coins(n);

    for(int i = 0; i < n; i++) cin >> coins[i];

    cin >> sum;

    vector<int> dp(sum + 1, 0);
    dp[0] = 1;

    for(int coin : coins){
        for(int j = coin; j <= sum; j++){
            dp[j] += dp[j - coin];
        }
    }

    cout << dp[sum];
    return 0;
}
```

Output

```
• PS S:\care\arrays_sheet_1> g++ Q16_CoinChange.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
3
1 2 3
4
4
❖ PS S:\care\arrays_sheet_1> █
```

17. Given an array of items, the i'th index element denotes the item id's and given a number m, the task is to remove m elements such that there should be minimum distinct id's left. Print the number of distinct id's.

Example 1 -

Input:

n = 6

arr[] = {2, 2, 1, 3, 3, 3}

m = 3

Output:

1

Explanation :

Remove 2,2,1

Example 2 -

Input:

n = 8

arr[] = {2, 4, 1, 5, 3, 5, 1, 3}

m = 2

Output:

3

Explanation:

Remove 2,4

Your Task:

This is a function problem. You don't have to take any input. Your task is to complete the distinctIds() which takes sorted array, its size n, and m as its parameter. You only need to find the minimum number of distinct IDs and return it. The driver code will print the returned value.

Expected Time Complexity: $O(n \log(n))$

Expected Auxillary Space: $O(n)$

Constraints:

$1 \leq n \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^6$

$1 \leq m \leq 10^3$

Code:

```
#include <iostream>
#include <vector>
#include <unordered_map>
#include <algorithm>
using namespace std;

int main() {
    int n, m;
    cin >> n;

    vector<int> arr(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    cin >> m;

    unordered_map<int,int> freq;

    for(int x : arr) freq[x]++;

    vector<int> counts;
    for(auto p : freq)
        counts.push_back(p.second);

    sort(counts.begin(), counts.end());
    int distinct = counts.size();
    for(int x : counts){
        if(m >= x){
            m -= x;
            distinct--;
        }
    }
    cout << distinct;
    return 0;
}
```

Output

```
• PS S:\care\arrays_sheet_1> g++ Q17_MinDistinctIDs.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
6
2 2 1 3 3 3
3
1
PS S:\care\arrays_sheet_1> |
```

18. A sorted array of distinct elements `arr[]` is rotated at some unknown point, the task is to find the minimum element in it.

Examples:

Input: `arr[] = [5, 6, 1, 2, 3, 4]`

Output: 1

Explanation: 1 is the minimum element in the array.

Input: `arr[] = [3, 1, 2]`

Output: 1

Explanation: Here 1 is the minimum element.

Input: `arr[] = [4, 2, 3]`

Output: 2

Explanation: Here 2 is the minimum element.

Constraints:

$1 \leq \text{arr.size()} \leq 10^6$

$1 \leq \text{arr}[i] \leq 10^9$

Code:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> arr(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    int low = 0, high = n - 1;

    while(low < high){
        int mid = (low + high) / 2;

        if(arr[mid] > arr[high])
            low = mid + 1;
        else
            high = mid;
    }

    cout << arr[low];
    return 0;
}
```

Output

```
• PS S:\care\arrays_sheet_1> g++ Q18_MinInRotatedSorted.cpp
• PS S:\care\arrays_sheet_1> ./a.exe
6
5 6 1 2 3 4
1
❖❖ PS S:\care\arrays_sheet_1> 
```


19. Given N numbers in an array, your task is to keep at most the top K numbers with respect to their frequency.

In other words, you have to iterate over the array, and after each index, determine the top K most frequent numbers until that iteration and store them in an array in decreasing order of frequency. An array will be formed for each iteration and stored in an array of arrays. If the total number of distinct elements is less than K, then keep all the distinct numbers in the array. If two numbers have equal frequency, place the smaller number first in the array.

Example 1:

Input:

N=5, K=4

arr[] = {5, 2, 1, 3, 2}

Output:

5

2 5

1 2 5

1 2 3 5

2 1 3 5

Explanation:

Firstly there was 5 whose frequency is max till now. So resulting sequence is {5}.

Then came 2, which is smaller than 5 but their frequencies are same so resulting sequence is {2, 5}.

Then came 1, which is the smallest among all the numbers arrived, so resulting sequence is {1, 2, 5}.

Then came 3, so resulting sequence is {1, 2, 3, 5}

Then again 2, which has the highest frequency among all numbers, so resulting sequence {2, 1, 3, 5}.

Example 2:

Input:

N=6, K=3

arr[] = {2, 1, 2, 1, 2, 1}

Output:

2

1 2

2 1

1 2

2 1

1 2

Explanation:

As total number of distinct values never exceeds 2, you have to return only those two values. In the case where frequency of 1 gets equal with the frequency of 2, you have to keep 1 before 2 in the array.

Your Task:

Since, this is a function problem. You don't need to take any input, as it is already accomplished by the driver code. You just need to complete the function `kTop()` that takes array `a`, integer `n` and integer `k` as parameters and returns the array of arrays.

Expected Time Complexity: $O(N \cdot K)$.

Expected Auxiliary Space: $O(N)$.

Constraints:

$$1 \leq N \leq 10^4$$

$$1 \leq K \leq 10^2$$

$$1 \leq a[i] \leq 10^2$$

Code:

```
#include <bits/stdc++.h>
using namespace std;

bool cmp(pair<int,int> a, pair<int,int> b){
    if(a.second == b.second)
        return a.first < b.first;
    return a.second > b.second;
}

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> arr(n);
    unordered_map<int,int> freq;
    for(int i = 0; i < n; i++)
        cin >> arr[i];

    for(int i = 0; i < n; i++){
        freq[arr[i]]++;

        vector<pair<int,int>> temp;

        for(auto p : freq)
            temp.push_back(p);

        sort(temp.begin(), temp.end(), cmp);

        for(int j = 0; j < min(k, (int)temp.size()); j++)
            cout << temp[j].first << " ";

        cout << endl;
    }
    return 0;
}
```

Output

```
● PS S:\care\arrays_sheet_1> g++ Q19_TopKFrequent.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
5 4
5 2 1 3 2
5
2 5
1 2 5
1 2 3 5
2 1 3 5
PS S:\care\arrays_sheet_1> █
```

20. Given an integer array `arr[]` and an integer `k`, your task is to find and return the `k`th smallest element in the given array.

Note: The `k`th smallest element is determined based on the sorted order of the array.

Examples :

Input: `arr[] = [10, 5, 4, 3, 48, 6, 2, 33, 53, 10]`, `k = 4`

Output: 5

Explanation: 4th smallest element in the given array is 5.

Input: `arr[] = [7, 10, 4, 3, 20, 15]`, `k = 3`

Output: 7

Explanation: 3rd smallest element in the given array is 7.

Constraints:

$1 \leq \text{arr.size()} \leq 10^5$

$1 \leq \text{arr}[i] \leq 10^5$

$1 \leq k \leq \text{arr.size()}$

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n, k;
    cin >> n;

    vector<int> arr(n);

    for(int i = 0; i < n; i++) cin >> arr[i];

    cin >> k;

    sort(arr.begin(), arr.end());

    cout << arr[k - 1];
    return 0;
}
```

Output

```
● PS S:\care\arrays_sheet_1> g++ Q20_KthSmallest.cpp
● PS S:\care\arrays_sheet_1> ./a.exe
6
7 10 4 3 20 15
3
7
PS S:\care\arrays_sheet_1> █
```